Informe de Implementación - Fase 2 (Canal Seguro TLS)

Elaborado por: Heider (para Cata)

Hola Cata, aquí tienes el resumen técnico de todo lo que se implementó en la **Fase 2** para que puedas armar el documento final del laboratorio según los requisitos del profesor Jeisson.

1. Patrón Arquitectónico Implementado

Se implementó el patrón de **Canal Seguro (Secure Channel Pattern)**.

- **Escenario de calidad abordado:** Confidencialidad e Integridad. Se asegura que la comunicación entre el navegador (frontend) y la API (backend) viaje cifrada, impidiendo que ataques de *man-in-the-middle* o interceptación de paquetes (como hicimos en la Fase 1) puedan leer el contenido de los mensajes (el texto plano).

2. Pasos de Implementación y Certificados (OpenSSL)

Se utilizó la herramienta `openssl` para crear nuestra propia Autoridad Certificadora (CA) y firmar los certificados del servidor.

Comandos utilizados:

```
# 1. Crear CA privada
openssl genrsa -out ca.key 2048
openssl req -x509 -new -nodes -key ca.key -sha256 -days 365 -out ca.crt -subj
"/C=CO/ST=Bogota/L=Bogota/O=EquipoLab5/CN=MiPropiaCA"

# 2. Crear clave y CSR del servidor
openssl genrsa -out server.key 2048
openssl req -new -key server.key -out server.csr -subj
"/C=CO/ST=Bogota/L=Bogota/O=EquipoLab5/CN=localhost"

# 3. Firmar certificado del servidor incluyendo SAN (Subject Alternative Name)
echo "subjectAltName=DNS:localhost,DNS:backend,DNS:frontend,IP:127.0.0.1" > san.ext
openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out
server.crt -days 365 -sha256 -extfile san.ext
```

3. Fragmentos de Código Relevantes

Backend (server.js)

Se modificó el servidor de Node.js para que use el módulo `https` en lugar de `http`, cargando los certificados generados. Además, se cumplió con el requisito de rechazar peticiones HTTP planas.

```
const https = require('node:https');
const fs = require('node:fs');

// Cargar certificados
```

```
const options = {
  key: fs.readFileSync('/certs/server.key'),
  cert: fs.readFileSync('/certs/server.crt')
};

// Iniciar servidor seguro
https.createServer(options, requestHandler).listen(8080, '0.0.0.0', () => { ... });

// Servidor secundario para rechazar HTTP plano (Requisito de la guía)
http.createServer((req, res) => {
    responder(res, 403, { error: 'Acceso denegado: Se requiere conexión HTTPS segura
(TLS).' });
}).listen(8081, '0.0.0.0', () => { ... });
```

Frontend (nginx.conf)

El frontend web se sirve con `nginx:alpine`. Se configuró para escuchar en el puerto 443 (SSL) e inyectar los certificados.

```
server {
    listen 443 ssl;
    server_name localhost;

    ssl_certificate /certs/server.crt;
    ssl_certificate_key /certs/server.key;

    location / {
        root   /usr/share/nginx/html;
        index  index.html index.htm;
    }
}
```

4. Resultados Observados

- La API y el Frontend ahora se comunican exclusivamente por `https://`.
- Un intento de usar HTTP plano (por ejemplo al puerto 8081) arroja un error 403 (Acceso denegado).
- Una petición sin verificar el certificado (o sin instalar nuestra `ca.crt` en el sistema) es rechazada por herramientas como `curl` debido al error `unable to get local issuer certificate`.
- La captura de paquetes (Fase 2) muestra tráfico cifrado (`Application Data`) bajo el protocolo TLS.

5. Recomendaciones (Para el informe del profesor)

- **Gestión de certificados:** Se recomienda automatizar la renovación de certificados utilizando herramientas como Let's Encrypt / Certbot en un entorno de producción, en lugar de gestionar una CA manual.
- **Almacenamiento seguro:** Las claves privadas (`.key`) nunca deben subirse al control de versiones en un proyecto real (aquí se incluyen por propósitos académicos). Deben inyectarse mediante servicios de gestión de secretos como AWS Secrets Manager o Docker Secrets.